TOY Banking SYSTEM
최종 코드 개선 보고서

C 언어 계좌 관리와 TXT 파일 입출력 검증

2026 년 10 월 12 일

이 프로그램은 구조체 배열과 기본 파일 입출력으로 계좌 정보를 관리하는 C 언어 학습 프로젝트이다. 이번 작업에서는 기존 함수와 메뉴 흐름을 유지하면서 Windows API 를 제거하고, 파일 저장 실패 때 기존 계좌 정보를 지킬 수 있도록 저장 절차를 수정했다.

파일 읽기와 저장, 입금과 출금, 재실행 후 잔액 확인을 중심으로 기본 기능을 검증했다. 보고서에 정리한 여섯 가지 테스트는 모두 통과했다. 원본 고객 정보 두 파일은 변경하지 않고 복사본으로 검사했다.

이체는 송금 계좌에서 돈을 빼는 기존 기능이며 수취 계좌에 입금하지 않는다. 작업 내역은 현재 로그인 세션의 메모리에만 남는다. 표준 C 파일 이름 변경을 여러 번 사용하므로 저장 전체의 원자성이나 정전 시 복구를 보장하지는 않는다.

목차

1 프로그램 개발 목적 및 개요

2 프로그램 요구사항

3 프로그램 주요 기능

4 프로그램 구조 및 자료흐름도

5 주요 소스 코드 및 함수 설명

6 파일 입출력 기능 구현

7 테스트 케이스 및 검증 과정

8 실제 프로그램 실행 결과

9 오류 원인과 해결 과정

10 결론 및 향후 개선 방향

분석 기준 main 커밋 9ee4d70defa641cf1072a3b51cb98be6902fb8b4

저장소 https://github.com/Seongppak/TOY-Banking-SYSTEM

1 프로그램 개발 목적 및 개요

계좌 검색, 인증, 입금, 출금, 잔액조회와 사용자 등록을 직접 구현하면서 함수, 구조체, 배열, 포인터 및 파일 입출력의 사용 방법을 익히는 것이 개발 목적이다. 계좌 정보는 프로그램 시작 때 customer_info.txt 에서 읽고, 등록이나 거래가 성공하면 전체 계좌를 다시 저장한다.

분석 대상은 최신 main 의 main.c, bandking.c, banking.h 와 Visual Studio 프로젝트이다. 이전 학습 단계별 C 파일은 빌드에서 제외된 상태로 보존했다. 별도의 기존 Word 또는 PDF 보고서는 작업 폴더와 저장소에서 발견되지 않아 최종 소스에 맞춘 보고서를 새로 작성했다.

2 프로그램 요구사항

구분	구현 기준
계좌 자료형	계좌번호 long long, 비밀번호 short, 이름 char[20], 잔액 long long
계좌 수와 이름	최대 100 개; 공백 없는 CP949 이름 최대 19 바이트
정상 파일	각 줄은 계좌번호 비밀번호 이름 잔액 네 필드
파일 오류	파일 없음, 잘못된 줄, 중복 계좌, 초과 계좌는 전체 읽기 실패
금액 검사	입금은 양수와 합계 범위 확인; 출금은 양수이며 잔액 이하
등록 취소	중복 계좌는 즉시 지정된 메시지를 출력하고 시작 메뉴로 복귀
저장 실패	거래 잔액 복원 또는 등록 수 유지; 성공 메시지와 로그 추가 없음
환경	Windows MSVC v145, C17, 소스 TXT 콘솔 모두 CP949

3 프로그램 주요 기능

시작 메뉴에서 1 번은 기존 계좌 로그인, 2 번은 신규 사용자 등록, 3 번은 저장 후 종료이다. 로그인에는 계좌 검색과 비밀번호 확인이 필요하다. 출금, 이체와 잔액조회는 다시 비밀번호를 확인한다. 입금은 로그인 이후 금액을 입력한다.

업무가 성공하면 작업 내역 확인 여부를 묻는다. 네를 고르면 이름을 확인한 후 내역을 출력하고, 아니오는 바로 시작 메뉴로 돌아간다. 다음 로그인 때 로그 개수는 0 으로 초기화된다. 따라서 하나의 로그인에서 여러 성공 거래를 이어 가거나 이전 로그인 기록을 누적 조회하는 흐름은 아니다.

4 프로그램 구조 및 자료흐름도

main.c 는 메뉴, 로그인과 거래 후 저장을 담당한다. bandking.c 는 계좌 검색, 인증, 금액 처리, 로그와 파일 입출력 함수를 정의한다. banking.h 는 구조체, 상수와 함수 선언을 공유한다. 계좌별 잔액은 human_list[i].balance 에 저장하여 다른 계좌 잔액과 섞이지 않도록 한다.

4 1 DFD Level 0



그림 1 외부 사용자와 계좌 파일 사이의 데이터 흐름

사각형은 외부 사용자, 둥근 상자는 처리 과정, 두 가로선은 데이터 저장소이다. 화살표는 메뉴 이동 순서가 아니라 실제 전달되는 자료이다. 사용자에게 받은 인증 정보와 거래 금액을 처리하고, 저장된 계좌를 읽거나 최신 전체 계좌를 저장한다. 수취 계좌에 돈을 보내는 화살표는 구현에 없으므로 표시하지 않았다.

D1 은 논리적인 계좌 정보 저장소이다. 저장 중 만들어지는 .tmp 와 .bak 는 D1 을 안전하게 갱신하기 위한 보조 파일이며 별도의 고객 계좌 데이터 흐름으로 추가하지 않았다.

4 2 DFD Level 1

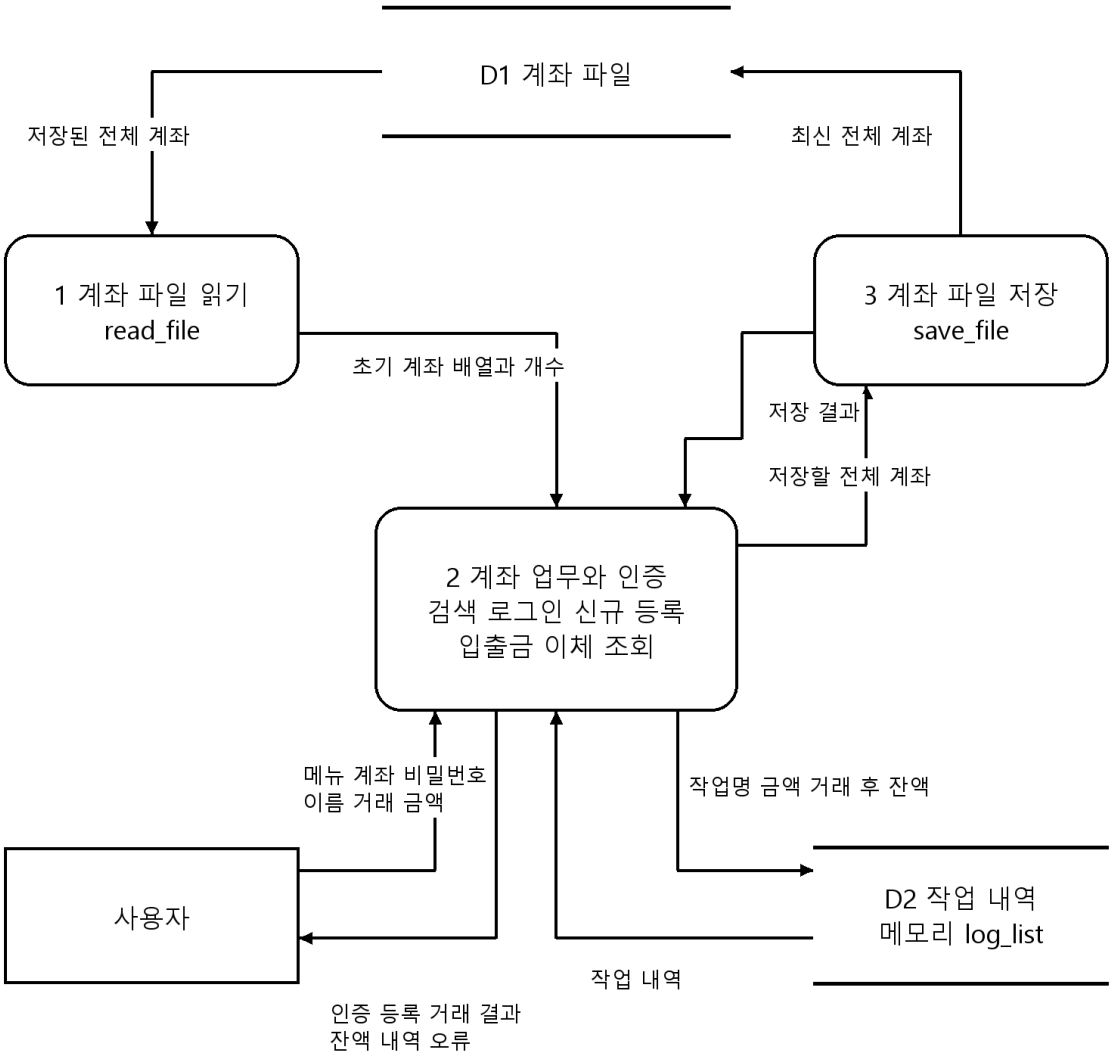


그림 2 파일 읽기 계좌 업무 파일 저장과 작업 내역의 관계

Level 0 의 사용자 입력과 출력, D1 에서 읽는 전체 계좌 및 D1 에 쓰는 최신 전체 계좌는 Level 1 에서도 같다. 2 번 과정 내부의 human_list 배열에서 검색, 신규 등록과 잔액 변경을 수행한다. 저장 결과는 호출한 업무로 돌아오며 실패하면 변경을 취소한다. 조회 로그는 파일이 아니라 log_list 배열에 저장된다.

4 3 프로그램 실행 순서

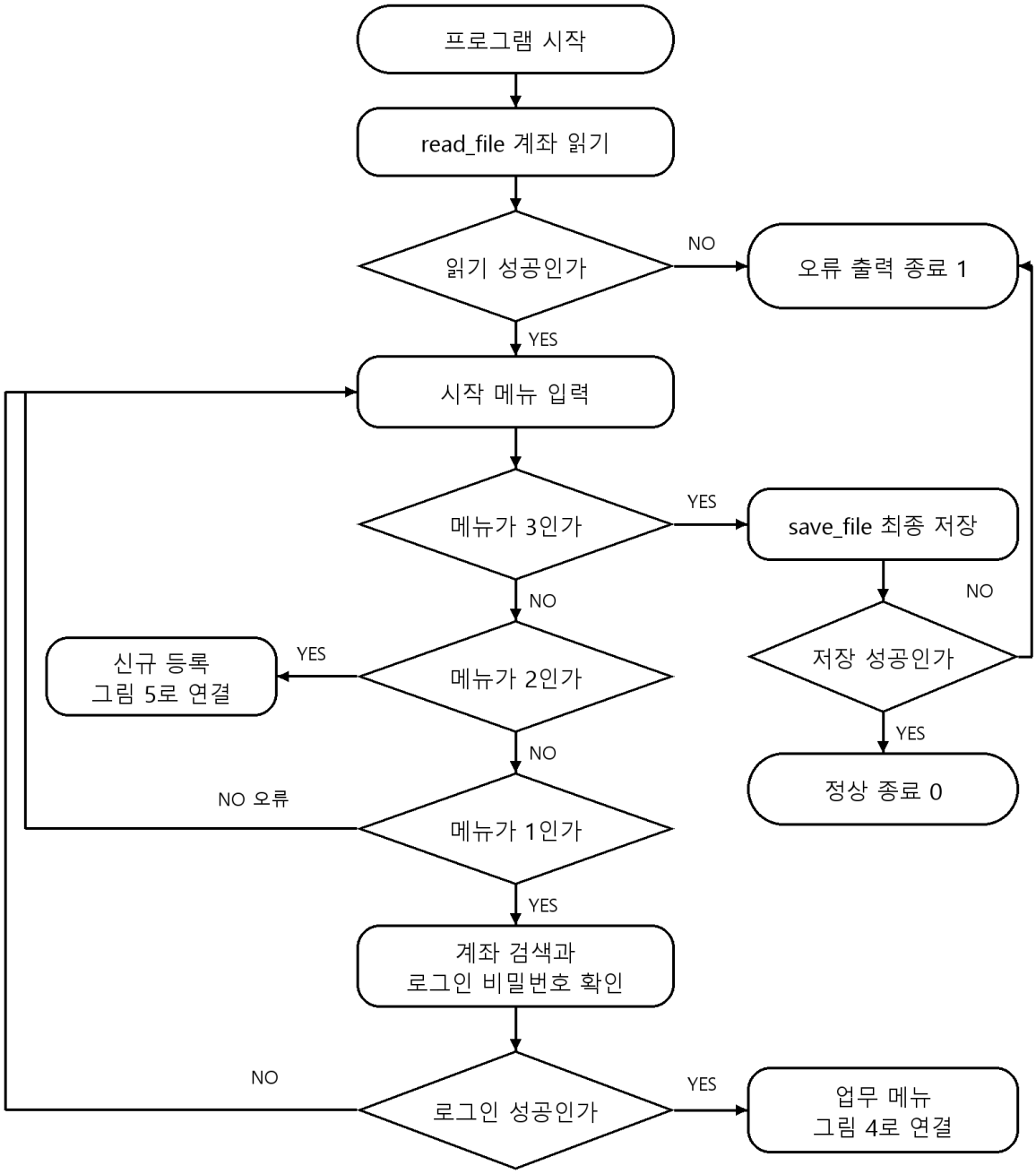


그림 3 주요 실행 순서와 YES NO 판단

잘못된 시작 메뉴는 다시 시작 메뉴를 묻는다. 업무 메뉴에서 금액이나 입력이 잘못되거나 저장이 실패하면 업무 메뉴를 다시 묻는다. 메인메뉴 또는 초기화를 입력하면 시작 메뉴로 돌아간다. 입력 종료 EOF 도 최종 저장과 종료 경로로 이동한다.

4 4 업무 처리 순서

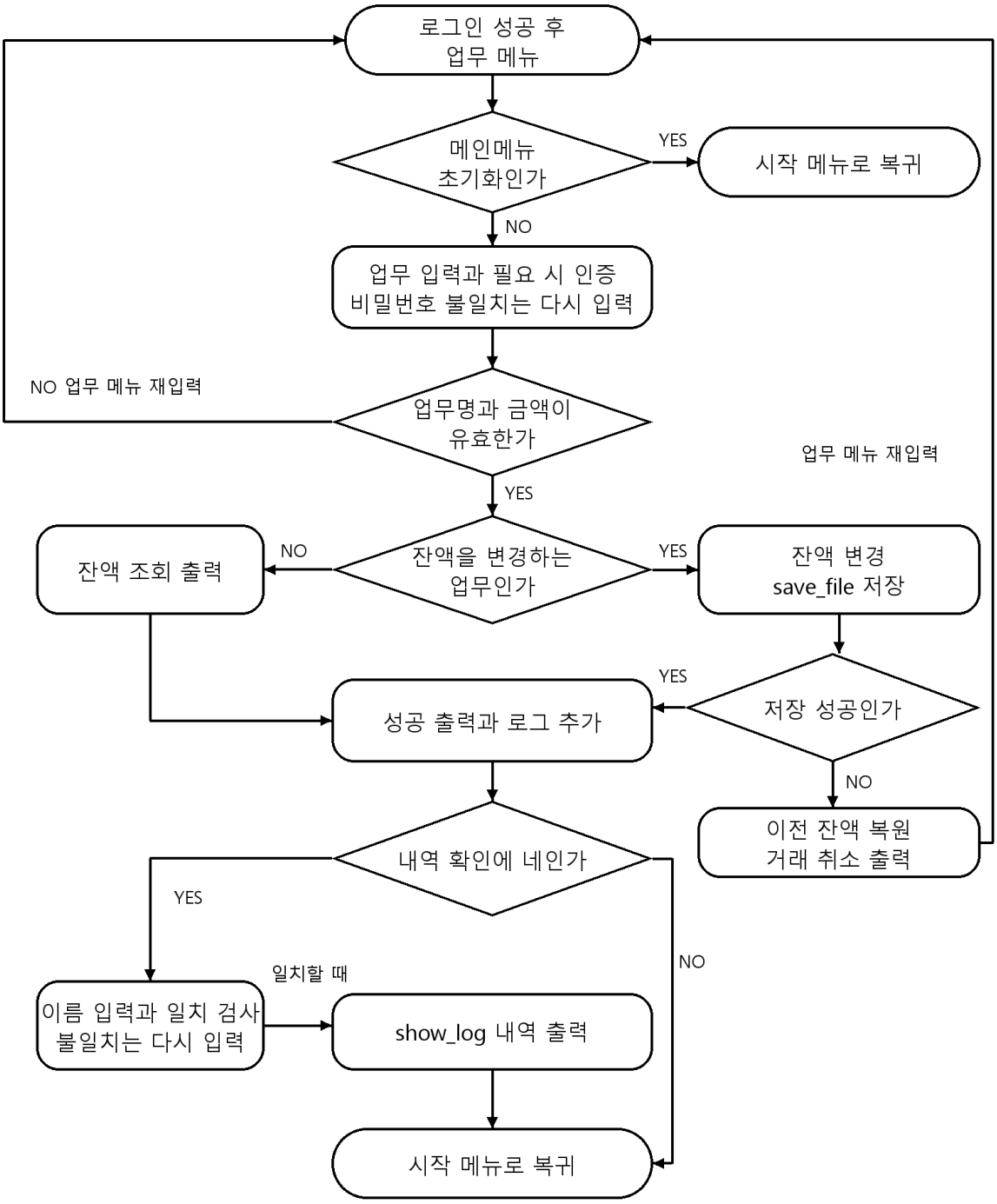


그림 4 업무 처리와 저장 실패 및 내역 조회 흐름

잔액조회는 계좌 파일을 변경하지 않고 조회 로그를 추가한다. 출금과 이체는 비밀번호를 다시 확인하며 틀린 경우 재입력을 받는다. 인증 입력의 EOF 는 최종 저장과 종료 경로로 이동한다. 이름 확인에서도 틀리면 다시 입력받는다.

4 5 신규 등록 순서

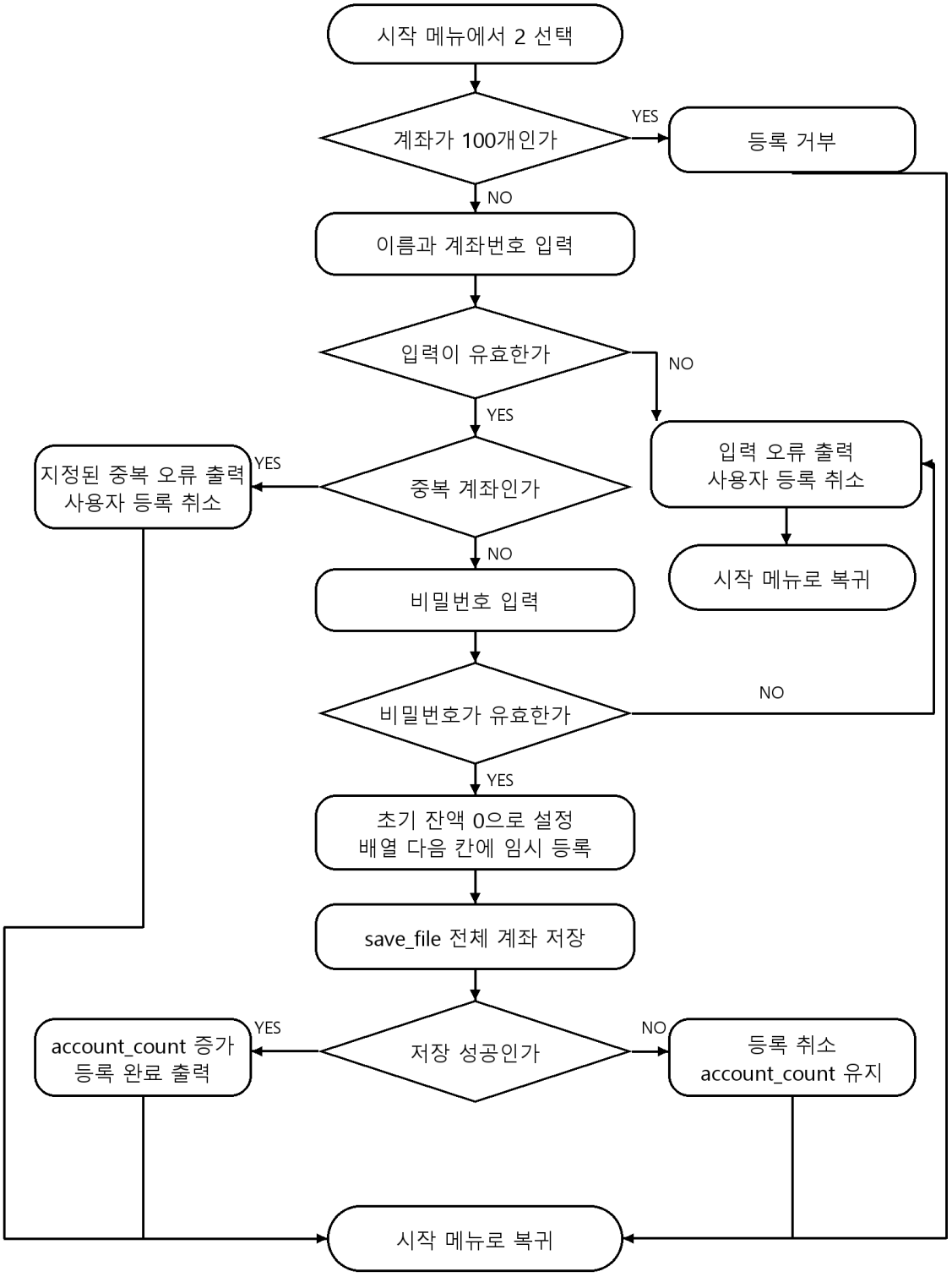


그림 5 신규 등록과 중복 계좌 취소 흐름

잘못된 입력과 중복 계좌는 등록을 취소하고 시작 메뉴로 돌아간다. 중복 판단의 YES 분기에서는 비밀번호를 추가로 입력받지 않는다. 임시 등록 칸은 저장 성공 후에만 account_count 에 포함된다.

5 주요 소스 코드 및 함수 설명

연한 노란색은 요청한 파일 입출력과 잔액 및 등록 확인 지점을 표시한다. 최신 main 에는 read_file 호출, 계좌별 balance 와 중복 검사 등이 이미 있었으므로 이 강조를 모두 이번에 새로 작성한 코드라고 해석하면 안 된다. 이번에 실제 수정한 부분은 Windows API 제거, 입력 줄 처리, 파일 저장 절차, 헤더 보호, C 빌드 설정, 솔루션 경로와 이체 안내이다.

5 1 구조체와 계좌별 잔액

```
struct human
{
    long long ac_num; // 등록되어 있는 계좌번호
    short pw; // 등록되어 있는 비밀번호
    char name[N]; // 등록되어 있는 이름
    long long balance;
};
```

계좌번호는 int 보다 큰 값을 담기 위해 long long 을 사용한다. 비밀번호는 0 부터 SHRT_MAX 인 32767 까지 검사한 후 short 로 저장한다. 이름 배열에는 마지막 널 문자까지 들어가야 하므로 실제 이름은 19 바이트까지 허용한다. 각 계좌의 잔액은 구조체 내부의 balance 에 둔다. 이 구조는 분석 기준 main 에도 이미 존재했다.

5 2 프로그램 시작과 파일 읽기

```
    account_count = read_file(file_name, human_list, MAX_ACCOUNTS);
    if (account_count < 0)
    {
        printf("계좌 파일을 읽을 수 없거나 데이터 형식이 잘못되었습니다.\n");
        return 1;
    }
    if (account_count == 0) printf("등록된 계좌가 없습니다. 신규 사용자를 등록하십시오.\n");
```

read_file 은 성공하면 계좌 개수를 반환한다. 0 은 빈 파일이고 -1 은 오류이다. 오류이면 메뉴에 들어가지 않고 종료 코드 1 로 끝나므로 불완전한 계좌 정보가 저장 함수로 전달되지 않는다.

5 3 계좌 검색과 금액 처리 함수

find_account 계좌번호를 순서대로 검색하여 인덱스를 반환한다. 없으면 -1 이다.

check_password 선택한 계좌의 pw 와 입력 비밀번호가 같은지 확인한다.

deposit 포인터로 전달된 잔액에 입금액을 더한다. main 에서 양수와 합계를 검사한다.

withdraw 0 이하 또는 잔액 초과면 0 을 반환한다. 차감 성공이면 1 이다.

transfer 출금처럼 차감한다. 수취 계좌의 입금은 처리하지 않는다.

show_balance 선택한 계좌의 잔액을 출력한다.

check_name 선택한 계좌의 이름과 입력 이름을 strcmp 로 비교한다.

save_log show_log 최대 20 개 내역을 메모리 배열에 기록하고 출력한다.

5 4 read_file 정의와 오류 처리

```
int read_file(const char *file_name, struct human human_list[], int max_accounts)
{
    FILE *file;
    struct human loaded[MAX_ACCOUNTS];
    char line[256], ac_num[64], pw[64], name[256], balance[64], extra;
    long long password;
    int count = 0, result, i;
    if (file_name == NULL || human_list == NULL || max_accounts <= 0 || max_accounts > MAX_ACCOUNTS)
        return -1;
    file = fopen(file_name, "rb");
    if (file == NULL) return -1;
    while ((result = read_line(file, line, sizeof(line))) != -1)
    {
        if (result != 1 || count >= max_accounts ||
            sscanf(line, "%63s %63s %255s %63s %c", ac_num, pw, name, balance, &extra) != 4 ||
            strlen(name) >= N || !parse_number(ac_num, &loaded[count].ac_num) ||
            !parse_number(pw, &password) || !parse_number(balance, &loaded[count].balance) ||
            loaded[count].ac_num <= 0 || password < 0 || password > SHRT_MAX || loaded[count].balance < 0)
            goto read_error;
        for (i = 0; i < count; i++)
            if (loaded[i].ac_num == loaded[count].ac_num) goto read_error;
        loaded[count].pw = (short)password;
        strcpy(loaded[count].name, name);
        count++;
    }
    if (ferror(file)) goto read_error;
    if (fclose(file) != 0) return -1;
    memcpy(human_list, loaded, count * sizeof(loaded[0]));
    return count;


read_error:
    fclose(file);
    return -1;
}
```

파일은 loaded 임시 배열에 먼저 읽는다. 한 줄을 네 문자열과 추가 문자로 나누어 필드 수를 확인하고, 숫자 변환과 범위를 검사한다. 중복 검사와 전체 읽기가 끝난 후에만 human_list 로 복사한다. 오류가 발생하면 파일을 닫고 -1 을 반환하여 호출한 배열을 바꾸지 않는다.

파일은 rb 모드로 읽는다. Windows 텍스트 모드에서 Ctrl Z 바이트가 파일 끝처럼 처리되어 뒤의 잘못된 자료가 숨겨지는 것을 막기 위해서이다. 별도 복사본의 정상 계좌 뒤 Ctrl Z 와 잘못된 줄을 붙여 전체 읽기 실패와 배열 보존을 추가 확인했다. CRLF 는 read_line 에서 줄 끝의 CR 을 제거하여 처리한다.

CP949 로 통일하기 위해 UTF-8 BOM 을 허용하던 처리를 제거했다. BOM 만 건너뛰어도 이름을 CP949 로 변환하는 것은 아니기 때문이다. 이름 인코딩을 자동 판별하거나 변환하는 기능은 추가하지 않았다. TXT 를 UTF-8 로 바꾸지 않는 것이 실행 전 조건이다.

5 5 save_file 의 쓰기 검증

```
int save_file(const char *file_name, const struct human human_list[], int account_count)
{
    FILE *file;
    char temporary[1024], backup[1024];
    int i, failed = 0;
    if (file_name == NULL || human_list == NULL || account_count < 0 || account_count > MAX_ACCOUNTS ||
        strlen(file_name) + 5 > sizeof(temporary)) return -1;
    sprintf(temporary, "%s.tmp", file_name);
    sprintf(backup, "%s.bak", file_name);
```

계좌 수와 파일 이름 길이를 확인한 뒤 .tmp 와 .bak 이름을 만든다. 이후 원본을 열 수 있는지 검사하고 남은 보조 파일이 있으면 덮어쓰지 않고 저장을 중단한다. 보조 파일의 내용 확인과 복구는 사람이 별도 복사본을 만든 후 진행한다.

```
    file = fopen(temporary, "w");
    if (file == NULL) return -1;
    for (i = 0; i < account_count; i++)
    {
        if (fprintf(file, "%lld %hd %s %lld\n", human_list[i].ac_num,
            human_list[i].pw, human_list[i].name, human_list[i].balance) < 0)
        {
            failed = 1;
            break;
        }
    }
    if (fflush(file) != 0) failed = 1;
    if (fclose(file) != 0) failed = 1;
```

모든 계좌의 현재 잔액과 신규 정보를 임시 파일에 쓴다. fprintf 실패와 버퍼 출력 fflush, 파일 닫기 fclose 실패를 함께 검사한다. 쓰기 검증에 실패하면 원본의 이름을 변경하지 않는다.

5 6 입력 함수

read_line 은 fgetc 로 한 줄을 읽으며 배열 길이를 넘은 줄의 나머지도 끝까지 소비한다. 중간 널 문자가 있으면 잘못된 줄로 표시한다. read_text 는 앞뒤 공백을 정리하고 공백이 포함된 이름이나 너무 긴 문자열을 거부한다. parse_number 는 strtoll 의 종료 위치와 ERANGE 를 확인하여 문자 섞임과 정수 범위 초과를 거부한다. read_password 는 short 범위를 추가로 확인한다. 이 검사들은 숫자 입력을 scanf 하나로 바꾸면 놓치기 쉬워 유지했다.

5 7 파일 교체와 복구

```
    if (failed || rename(file_name, backup) != 0)
    {
        remove(temporary);
        return -1;
    }
    if (rename(temporary, file_name) != 0)
    {
        if (rename(backup, file_name) != 0)
            printf("기존 파일 복구에 실패했습니다. %s 와 %s 를 보존하고 확인하십시오.\n", backup, temporary);
        else remove(temporary);
        return -1;
    }
    if (remove(backup) != 0)
        printf("저장은 완료되었으나 백업 파일을 지우지 못했습니다. %s 를 확인하십시오.\n", backup);
    return 0;
}
```

원본을 .bak 로 옮긴 다음 검증된 .tmp 를 원본 이름으로 바꾼다. 두 번째 rename 이 실패하면 백업을 되돌린다. 백업 복구까지 실패하면 .bak 와 .tmp 를 지우지 않는다. 마지막 백업 삭제만 실패한 경우에는 새 파일이 이미 저장되었으므로 저장 성공을 반환하면서 정리 실패를 알린다.

5 8 거래 저장 실패와 잔액 복원

```
                previous_balance = human_list[account_index].balance;
                deposit(&human_list[account_index].balance, in_money);
                if (save_file(file_name, human_list, account_count) != 0)
                {
                    human_list[account_index].balance = previous_balance;
                    printf("파일 저장에 실패하여 거래가 취소되었습니다.\n");
                    continue;
                }
```

입금 전 잔액을 previous_balance 에 보관한다. 파일 저장이 실패하면 메모리 잔액을 되돌리고 거래를 취소한다. 성공 메시지와 save_log 는 저장 성공 이후에만 실행된다. 출금과 이체도 같은 저장 실패 처리를 사용한다.

5 9 신규 등록과 중복 계좌 확인

```
            if (find_account(human_list, account_count, new_user.ac_num) != -1)
            {
                printf("이미 등록된 계좌번호입니다.\n 사용자 등록이 취소되었습니다.\n\n 메인 메뉴로 돌아갑니다.\n");
                continue;
            }
            printf("등록할 비밀번호를 입력하십시오.:");
```

중복 계좌를 찾으면 지정된 취소 메시지를 출력하고 continue 로 시작 메뉴를 다시 표시한다. 같은 계좌번호를 다시 입력하도록 강제하지 않으며 비밀번호를 새로 묻지도 않는다.

```
            new_user.balance = 0;
            human_list[account_count] = new_user;
            if (save_file(file_name, human_list, account_count + 1) != 0)
            {
                printf("파일 저장에 실패하여 사용자 등록이 취소되었습니다.\n 메인 메뉴로 돌아갑니다.\n");
                continue;
            }
            account_count++;
```

신규 계좌는 초기 잔액 0 이다. 배열의 다음 칸에 정보를 넣어 전체 계좌를 저장하되, 저장 성공 때만 account_count 를 늘린다. 실패한 정보는 유효한 계좌 범위에 포함되지 않는다. 기존 계좌를 변경하지 않는지는 정상 등록, 중복 등록 취소와 등록 저장 실패 검사에서 검사했다.

5 10 정상 종료와 입력 종료

```
finish:
    if (save_file(file_name, human_list, account_count) != 0)
    {
        printf("계좌 파일 저장에 실패했습니다.\n");
        return 1;
    }
    return 0;
}
```

3 번 메뉴 또는 EOF 로 프로그램이 종료될 때도 전체 계좌 저장 결과를 확인한다. 최종 저장 실패이면 오류를 출력하고 종료 코드 1 을 반환한다. 파일 읽기 실패는 이 종료 지점에 도달하지 않아 잘못된 데이터로 원본을 덮어쓰지 않는다.

6 파일 입출력 기능 구현

6 1 데이터 형식과 바이트 검사

```
100152351124 1524 김백제 10000
100144523657 1275 이온조 25000
100142566328 1685 최쿠농 100000
```

프로젝트 바로 아래의 TXT 는 97 바이트이고, 하위 폴더의 기존 TXT 는 95 바이트이다. 두 파일은 계좌 데이터가 같으며 마지막 줄바꿈 유무가 다르다. CP949 로 엄격히 디코딩하면 김백제, 이온조, 최쿠농이 나온다. UTF-8 로 읽어 깨져 보이는 것은 원본 이름 자체가 손상되었다는 뜻이 아니다. 파일 바이트를 확인한 후 두 파일 모두 그대로 보존했다.

김백제의 CP949 바이트는 B1 E8 B9 E9 C1 A6 이다. 정상 계좌 파일 읽기 검사는 이 이름을 실제 C 함수 출력에서 확인했고, 신규 계좌 재실행 검사는 신규 한글 이름을 저장한 후 재실행하여 이름 인증까지 확인했다. 원본 파일 SHA-256 은 reports/validation/original_data_sha256.json 에 기록했다.

6 2 CP949 선택 이유와 Visual Studio 설정

기존 소스, 헤더, 계좌 TXT 가 이미 CP949 여서 이를 유지하면 데이터 변환 없이 기존 주석과 한글 입력을 보존할 수 있다. C 문자열은 바이트 배열이므로 소스의 실행 문자열, 입력 콘솔과 TXT 인코딩이 같아야 strcmp 와 이름 인증도 맞는다.

Visual Studio 프로젝트 네 구성에 /source-charset:.949 /execution-charset:.949 를 유지하고 CompileAsC 및 C17 을 명시했다. 헤더의 pragma once 는 표준 전처리기 헤더 보호로 바꾸었다. 콘솔 코드 페이지는 프로그램 밖에서 chcp 949 로 설정한다. run_cp949.cmd 에 EXE 전체 경로를 전달하면 949 설정 후 프로젝트 폴더에서 실행한다. 테스트에는 이 폴더와 TXT 의 복사본을 사용한다.

F5 또는 CTRL+F5 로 직접 실행할 때도 콘솔 입력과 출력이 949 인지 확인해야 한다. UTF-8 콘솔 설정을 그대로 쓰면 한글이 깨지거나 한글 명령 비교가 실패한다. IDE 에서 소스가 깨지면 한국어 코드 페이지 949 로 다시 열고, 한글을 지원하는 콘솔 글꼴을 사용한다. 프로그램이 콘솔 인코딩을 자동 변경하는 기능은 이제 없다.

6 3 저장의 안전성과 한계

임시 파일을 완성한 다음 이름을 바꾸므로 임시 파일 쓰기 실패가 바로 원본 손상으로 이어지지는 않는다. 그러나 원본→백업과 임시→원본은 별도 작업이다. 그 사이 강제 종료나 정전이 발생하면 원본 이름이 잠시 없을 수 있다. fflush 와 fclose 성공이 물리적인 디스크 저장을 강제로 보장하는 것은 아니다. 동시 실행, 자동 복구, 잠금과 자동 충돌 해결은 제공하지 않는다.

복구가 필요하면 프로그램을 종료하고 원본, .bak 와 .tmp 를 먼저 다른 폴더에 복사한다. 원본이 없다면 .bak 의 기존 자료와 .tmp 의 새 자료를 비교하여 선택한 복사본을 customer_info.txt 로 복원한다. 보조 파일은 확인 후 별도 보관 폴더로 옮긴다. 아직 확인하지 않은 파일을 지우거나 그대로 덮어쓰지 않는다.

7 테스트 케이스 및 검증 과정

파일 입출력과 기본적인 입금·출금 기능을 중심으로 여섯 가지 테스트를 정리했다. 각 검사는 원본 customer_info.txt 의 복사본으로 수행했다. 프로그램의 출력과 저장된 잔액을 비교하여 결과를 확인했으며 원본 고객 파일은 변경하지 않았다.

Windows 에서 MSVC 로 빌드한 실행 파일을 사용했다. Debug 와 Release 의 x64 및 Win32 구성은 모두 컴파일 오류와 경고 없이 빌드되었다. 아래에 정리한 기본 기능 검사는 모두 PASS 이다.

검증 방법

처음에는 김백제 계좌의 잔액 10000 원을 기준으로 검사했다. 입금과 출금 검사는 서로 다른 복사본에서 실행했다. 저장 검사는 거래 이후 TXT 내용을 확인하고, 재실행 검사는 새 프로세스에서 같은 파일을 다시 읽어 잔액을 조회했다.

7 1 기본 기능 테스트

정상 파일 읽기

테스트 기능	정상 파일 읽기
테스트 목적	정상 계좌 3 개와 CP949 이름을 확인
사전 조건	100152351124 1524 김백제 10000 100144523657 1275 이온조 25000 100142566328 1685 최쿠농 100000
실제 입력값	계좌 TXT 내용은 사전 조건과 같음
예상 결과	COUNT=3, 김백제 10000, 이온조 25000, 최쿠농 100000
실제 결과	COUNT=3 UNCHANGED=0; ACCOUNT=100152351124 1524 김백제 10000; ACCOUNT=100144523657 1275 이온조 25000; ACCOUNT=100142566328 1685 최쿠농 100000 파일 3 개 계좌; 첫 계좌 잔액 10000 원; 프로세스 종료 코드 0
판정	PASS
오류 예방 목적	파일 누락 읽기 및 한글 손상

없는 파일 처리

테스트 기능	없는 파일 처리
테스트 목적	파일 없음은 오류이며 새 파일을 만들지 않음
사전 조건	customer_info.txt 없음
실제 입력값	계좌 TXT 내용은 사전 조건과 같음
예상 결과	COUNT=-1, 배열 불변, 종료 코드 1, 파일 생성 안 함
실제 결과	COUNT=-1 UNCHANGED=1 계좌 파일 없음; 프로세스 종료 코드 1
판정	PASS
오류 예방 목적	없는 데이터의 정상 사용

7 1 기본 기능 테스트

입금

테스트 기능	입금
테스트 목적	10000 원 계좌에 5000 원 입금
사전 조건	100152351124 1524 김백제 10000 100144523657 1275 이온조 25000 100142566328 1685 최쿠농 100000
실제 입력값	1 → 100152351124 → 1524 → 입금 → 5000 → 아니오 → 3
예상 결과	잔액 15000, 나머지 계좌 불변
실제 결과	입금이 완료 되었습니다. 현재 잔액은 15000 원 입니다. 파일 3 개 계좌; 첫 계좌 잔액 15000 원; 프로세스 종료 코드 0
판정	PASS
오류 예방 목적	입금 계산 또는 대상 계좌 오류

출금

테스트 기능	출금
테스트 목적	10000 원 계좌에서 3000 원 출금
사전 조건	100152351124 1524 김백제 10000 100144523657 1275 이온조 25000 100142566328 1685 최쿠농 100000
실제 입력값	1 → 100152351124 → 1524 → 출금 → 1524 → 3000 → 아니오 → 3
예상 결과	잔액 7000, 다른 계좌 불변
실제 결과	비밀번호가 올바르게 입력되었습니다.; 출금이 완료 되었습니다. 현재 잔액은 7000 원 입니다. 파일 3 개 계좌; 첫 계좌 잔액 7000 원; 프로세스 종료 코드 0
판정	PASS
오류 예방 목적	출금 계산 오류

7 1 기본 기능 테스트

거래 후 파일 저장

테스트 기능	거래 후 파일 저장
테스트 목적	입금 직후 EOF 로 종료해도 갱신
사전 조건	100152351124 1524 김백제 10000 100144523657 1275 이온조 25000 100142566328 1685 최쿠농 100000
실제 입력값	1 → 100152351124 → 1524 → 입금 → 5000
예상 결과	파일 잔액 15000, .tmp/.bak 없음
실제 결과	입금이 완료 되었습니다. 현재 잔액은 15000 원 입니다. 파일 3 개 계좌; 첫 계좌 잔액 15000 원; 프로세스 종료 코드 0
판정	PASS
오류 예방 목적	종료 시 입금 데이터 유실

재실행 후 잔액 유지

테스트 기능	재실행 후 잔액 유지
테스트 목적	별도 프로세스에서 저장된 잔액 조회
사전 조건	100152351124 1524 김백제 15000 100144523657 1275 이온조 25000 100142566328 1685 최쿠농 100000
실제 입력값	1 → 100152351124 → 1524 → 잔액조회 → 1524 → 아니오 → 3
예상 결과	15000 출력
실제 결과	비밀번호가 올바르게 입력되었습니다.; 현재 귀하의 잔액은 15000 원 입니다. 파일 3 개 계좌; 첫 계좌 잔액 15000 원; 프로세스 종료 코드 0
판정	PASS
오류 예방 목적	메모리에만 저장되는 잔액

8 실제 프로그램 실행 결과

다음 화면은 Visual Studio 의 MSVC 로 빌드한 EXE 를 실제 실행하여 캡처한 표준 출력의 재현이다. IDE 화면 캡처가 아니다. 전송한 입력값을 원래 프롬프트 뒤에 삽입하고 흰 배경과 검은 고정폭 글꼴로 표시했다. 줄바꿈과 공백은 출력 기록에 맞추었으며 입력값은 굵게 표시한다. 출력과 입력 원본은 test_results.json 에 구분하여 기록되어 있다.

8 1 정상 로그인과 잔액조회

```
========== BANKING SYSTEM ==========


1. 기존 계좌 로그인
2. 신규 사용자 등록
3. 프로그램 종료


원하시는 작업을 선택하십시오.:1
등록하신 계좌번호를 입력하십시오.:100152351124
등록된 계좌의 비밀번호를 입력하십시오.:1524
원하시는 작업을 입력해주세요( 입금, 출금, 이체, 잔액조회, 메인메뉴 ):잔액조회
등록된 계좌의 비밀번호를 입력하십시오.:1524
비밀번호가 올바르게 입력되었습니다.
현재 귀하의 잔액은 10000 원 입니다.
작업내용을 확인 하시겠습니까?(네 또는 아니오):아니오
시작 메뉴로 돌아갑니다.


========== BANKING SYSTEM ==========


1. 기존 계좌 로그인
2. 신규 사용자 등록
3. 프로그램 종료


원하시는 작업을 선택하십시오.:3
```

실제 실행 출력과 입력 재현

8 실제 프로그램 실행 결과

8 2 입금 후 잔액 15000 원

```
========== BANKING SYSTEM ==========


1. 기존 계좌 로그인
2. 신규 사용자 등록
3. 프로그램 종료


원하시는 작업을 선택하십시오.:1
등록하신 계좌번호를 입력하십시오.:100152351124
등록된 계좌의 비밀번호를 입력하십시오.:1524
원하시는 작업을 입력해주세요( 입금, 출금, 이체, 잔액조회, 메인메뉴 ):입금
입금할 금액을 입력하여 주십시오.:5000
입금이 완료 되었습니다. 현재 잔액은 15000 원 입니다.
작업내용을 확인 하시겠습니까?(네 또는 아니오):아니오
시작 메뉴로 돌아갑니다.


========== BANKING SYSTEM ==========


1. 기존 계좌 로그인
2. 신규 사용자 등록
3. 프로그램 종료


원하시는 작업을 선택하십시오.:3
```

실제 실행 출력과 입력 재현

8 3 출금 후 잔액 7000 원

```
========== BANKING SYSTEM ==========


1. 기존 계좌 로그인
2. 신규 사용자 등록
3. 프로그램 종료


원하시는 작업을 선택하십시오.:1
등록하신 계좌번호를 입력하십시오.:100152351124
등록된 계좌의 비밀번호를 입력하십시오.:1524
원하시는 작업을 입력해주세요( 입금, 출금, 이체, 잔액조회, 메인메뉴 ):출금
등록된 계좌의 비밀번호를 입력하십시오.:1524
비밀번호가 올바르게 입력되었습니다.
출금할 금액을 입력하여 주십시오.:3000
출금이 완료 되었습니다. 현재 잔액은 7000 원 입니다.
작업내용을 확인 하시겠습니까?(네 또는 아니오):아니오
시작 메뉴로 돌아갑니다.


========== BANKING SYSTEM ==========


1. 기존 계좌 로그인
2. 신규 사용자 등록
3. 프로그램 종료


원하시는 작업을 선택하십시오.:3
```

실제 실행 출력과 입력 재현

8 실제 프로그램 실행 결과

8 4 신규 사용자 등록

```
========== BANKING SYSTEM ==========

1. 기존 계좌 로그인
2. 신규 사용자 등록
3. 프로그램 종료

원하시는 작업을 선택하십시오.:2
신규 사용자 이름을 입력하십시오.:홍길동
등록할 계좌번호를 입력하십시오.:100199999999
등록할 비밀번호를 입력하십시오.:1234
신규 사용자 등록이 완료되었습니다.

계좌번호: 100199999999
이름: 홍길동
초기 잔액: 0 원

메인 메뉴로 돌아갑니다.
========== BANKING SYSTEM ==========

1. 기존 계좌 로그인
2. 신규 사용자 등록
3. 프로그램 종료

원하시는 작업을 선택하십시오.:3
```

실제 실행 출력과 입력 재현

8 5 중복 계좌 즉시 취소

```
========== BANKING SYSTEM ==========

1. 기존 계좌 로그인
2. 신규 사용자 등록
3. 프로그램 종료

원하시는 작업을 선택하십시오.:2
신규 사용자 이름을 입력하십시오.:홍길동
등록할 계좌번호를 입력하십시오.:100152351124
이미 등록된 계좌번호입니다.
사용자 등록이 취소되었습니다.

메인 메뉴로 돌아갑니다.
========== BANKING SYSTEM ==========

1. 기존 계좌 로그인
2. 신규 사용자 등록
3. 프로그램 종료

원하시는 작업을 선택하십시오.:3
```

실제 실행 출력과 입력 재현

8 실제 프로그램 실행 결과

8 6 별도 프로세스 재실행 후 잔액 유지

```
========== BANKING SYSTEM ==========


1. 기존 계좌 로그인
2. 신규 사용자 등록
3. 프로그램 종료


원하시는 작업을 선택하십시오.:1
등록하신 계좌번호를 입력하십시오.:100152351124
등록된 계좌의 비밀번호를 입력하십시오.:1524
원하시는 작업을 입력해주세요( 입금, 출금, 이체, 잔액조회, 메인메뉴 ):잔액조회
등록된 계좌의 비밀번호를 입력하십시오.:1524
비밀번호가 올바르게 입력되었습니다.
현재 귀하의 잔액은 15000 원 입니다.
작업내용을 확인 하시겠습니까?(네 또는 아니오):아니오
시작 메뉴로 돌아갑니다.


========== BANKING SYSTEM ==========


1. 기존 계좌 로그인
2. 신규 사용자 등록
3. 프로그램 종료


원하시는 작업을 선택하십시오.:3
```

실제 실행 출력과 입력 재현

8 실제 프로그램 실행 결과

8 7 신규 계좌 재실행과 작업 내역 조회

```
========== BANKING SYSTEM ==========


1. 기존 계좌 로그인
2. 신규 사용자 등록
3. 프로그램 종료


원하시는 작업을 선택하십시오.:1
등록하신 계좌번호를 입력하십시오.:100199999999
등록된 계좌의 비밀번호를 입력하십시오.:1234
원하시는 작업을 입력해주세요( 입금, 출금, 이체, 잔액조회, 메인메뉴 ):잔액조회
등록된 계좌의 비밀번호를 입력하십시오.:1234
비밀번호가 올바르게 입력되었습니다.
현재 귀하의 잔액은 0원 입니다.
작업내용을 확인 하시겠습니까?(네 또는 아니오):네
계좌 소유주의 이름을 입력하십시오.:홍길동
실명 확인 되었습니다.


------ 작업 내역 ------
1. 잔액조회 / 현재 잔액 0원
시작 메뉴로 돌아갑니다.


========== BANKING SYSTEM ==========


1. 기존 계좌 로그인
2. 신규 사용자 등록
3. 프로그램 종료


원하시는 작업을 선택하십시오.:3
```

실제 실행 출력과 입력 재현

8 실제 프로그램 실행 결과

8 8 입금 출금 조회 작업 내역

입금·출금·잔액조회 연속 실행 기록에서 각 로그인과 거래 및 내역 부분을 발췌했다. 거래 사이에는 실제 시작 메뉴가 출력되며 계좌번호와 비밀번호를 다시 입력했다. 시작 메뉴를 반복해서 보여 주는 부분만 이 페이지에서 생략했다.

```
등록하신 계좌번호를 입력하십시오.:100152351124
등록된 계좌의 비밀번호를 입력하십시오.:1524
원하시는 작업을 입력해주세요( 입금, 출금, 이체, 잔액조회, 메인메뉴 ):입금
입금할 금액을 입력하여 주십시오.:5000
입금이 완료 되었습니다. 현재 잔액은 15000 원 입니다.
작업내용을 확인 하시겠습니까?(네 또는 아니오):네
계좌 소유주의 이름을 입력하십시오.:김백제
실명 확인 되었습니다.


------ 작업 내역 ------
1. 입금 / 5000 원 / 거래 후 잔액 15000 원
시작 메뉴로 돌아갑니다.
등록하신 계좌번호를 입력하십시오.:100152351124
등록된 계좌의 비밀번호를 입력하십시오.:1524
원하시는 작업을 입력해주세요( 입금, 출금, 이체, 잔액조회, 메인메뉴 ):출금
등록된 계좌의 비밀번호를 입력하십시오.:1524
비밀번호가 올바르게 입력되었습니다.
출금할 금액을 입력하여 주십시오.:3000
출금이 완료 되었습니다. 현재 잔액은 12000 원 입니다.
작업내용을 확인 하시겠습니까?(네 또는 아니오):네
계좌 소유주의 이름을 입력하십시오.:김백제
실명 확인 되었습니다.


------ 작업 내역 ------
1. 출금 / 3000 원 / 거래 후 잔액 12000 원
시작 메뉴로 돌아갑니다.
등록하신 계좌번호를 입력하십시오.:100152351124
등록된 계좌의 비밀번호를 입력하십시오.:1524
원하시는 작업을 입력해주세요( 입금, 출금, 이체, 잔액조회, 메인메뉴 ):잔액조회
등록된 계좌의 비밀번호를 입력하십시오.:1524
비밀번호가 올바르게 입력되었습니다.
현재 귀하의 잔액은 12000 원 입니다.
작업내용을 확인 하시겠습니까?(네 또는 아니오):네
계좌 소유주의 이름을 입력하십시오.:김백제
실명 확인 되었습니다.


------ 작업 내역 ------
1. 잔액조회 / 현재 잔액 12000 원
시작 메뉴로 돌아갑니다.
```

9 오류 원인과 해결 과정

문제	원인과 실제 수정
Windows 의존성	windows.h SetConsoleCP SetConsoleOutputCP MoveFileExA 와 _WIN32 분기를 제거; 콘솔 설정은 실행 CMD 로 이동
표준 C 파일 교체	Windows 에서 기존 파일 위 rename 이 실패할 수 있어 원본 백업 후 교체와 복구 절차 사용
혼합 인코딩	CP949 이름에 UTF-8 BOM 만 제거해도 변환되지 않음; CP949 전제를 README 와 보고서에 명시
입력 줄 경계	fgets 문자열 안 NUL 뒤 내용이 숨겨질 수 있음; 문자 반복으로 길이와 NUL 검출
C 빌드 설정	C++20 설정을 C17 과 C 컴파일로 수정; 경고 수준 W4
솔루션 경로	저장소 루트 slnx 가 개인 컴퓨터 source/repos 경로 참조; 저장소 내부 상대 경로로 수정
이체 완료 문구	수취 계좌 입금이 없는데 완료처럼 보일 수 있음; 차감 안내와 차감 완료 문구로 수정

한글이 깨지지 않도록 소스와 TXT 파일, 컴파일 설정을 CP949 로 맞췄다. 출력 비교에서는 Windows 의 줄바꿈 형식을 통일했다. 테스트마다 새 복사본을 사용하여 앞선 거래 결과가 다음 검사에 영향을 주지 않도록 했다.

10 결론 및 향후 개선 방향

기존 함수명, 주석, 구조체와 업무 메뉴를 유지하면서 Windows API 없이 C 표준 라이브러리로 입력과 계좌 파일 저장을 처리했다. 입출금과 등록의 저장 오류를 성공으로 표시하지 않는 흐름을 유지하고, 저장 교체 또는 복구 실패에 대비하여 기존 자료를 남기는 방법을 추가했다. 복잡한 설계나 불필요한 업무 함수 분리는 하지 않았다.

start_menu, finish 와 read_error 의 goto 는 여러 중첩 입력에서 메뉴 복귀, 종료 저장과 파일 닫기를 한 곳으로 모으는 역할이 있어 유지했다. 이번 범위에서 전부 없애면 기존 메뉴와 인증 흐름을 크게 다시 작성해야 하므로 변경 이점이 작았다.

앞으로 개선할 부분은 수취 계좌를 선택하고 양쪽 잔액을 함께 저장하는 실제 이체, 재실행 후에도 남는 거래 내역, 비밀번호의 안전한 저장이다. 현재 비밀번호는 평문 TXT 의 short 값이므로 실제 금융 서비스에 적합하지 않다. 동시 실행이나 정전 복구를 추가한다면 파일 잠금과 더 강한 저장 방법도 검토해야 한다.

추가 수동 확인 항목은 Visual Studio F5/CTRL+F5 콘솔의 949 설정, IME 한글 입력과 글꼴 표시이다. 이번 보고서의 실행 검사는 Windows MSVC 바이너리와 CP949 표준 입출력으로 수행했으며 IDE 창을 직접 캡처하지 않았다. 실제 정전과 물리적인 디스크 장애도 미실행이다.

재현 방법과 전체 실행 증거는 tests/README.md 및 reports/validation 에 둔다. 저장소에 임시 계좌 TXT, 실행 파일과 빌드 결과물을 커밋하지 않는다.